MYTH.EXE

ADVANCED SOFTWARE ENGINEERING — GROUP PROJECT

Name	University email
Samitha Piumanga Michael	s.michael@studenti.unipi.it
Abhinav Das	a.das6@studenti.unipi.it
Sara Xxx	s.xxx10@studenti.unipi.it
Ebin Neelankavil Lenin	e.neelankavillenin@studenti.unipi.it

CONTENTS

SECTION	TOPIC
1	Executive summary
2	Project background and objectives
3	Scope and requirements
4	System architecture
5	Microservices
6	API design and service communication
7	Game domain and rules
8	Implementation status
9	Testing and verification
10	Security, reliability and data management
11	Main API endpoints
12	Full user stories and acceptance criteria

1. EXECUTIVE SUMMARY

Myth.exe is a backend prototype for a monster-collection game. Players can register and authenticate, browse a monster catalogue, manage a collection, create teams and resolve encounters against wild monsters. The backend is divided into four independently deployed FastAPI microservices: User, Monster, Team and Encounter. Each service has a dedicated PostgreSQL database and is orchestrated through Docker Compose.

The User Service manages accounts and authentication; the Monster Service manages catalogue and collection data; the Team Service manages player teams and team power; and the Encounter Service calculates encounter outcomes, records history and requests monster rewards after successful encounters.

The latest supplied test evidence reports 51 passing automated tests and no failures. The API Gateway and graphical frontend are not implemented in the current prototype. HTTPS deployment and production security hardening are also future work.

2. PROJECT BACKGROUND AND OBJECTIVES

2.1 Background

The project applies microservice-oriented design to a small game domain. Instead of placing all backend functionality in one application, it separates user identity, monster data, team management and encounter processing into distinct services.
Each service exposes HTTP APIs and owns its own database.

2.2 Objectives

The main objectives are to provide player registration and login; expose a monster catalogue and player collections; allow teams to be created and managed; calculate encounter outcomes on the backend; persist successful and failed encounter history; award a wild monster after a successful encounter; and demonstrate service separation, API communication, containerisation and automated testing.

3. SCOPE AND REQUIREMENTS

3.1 Functional scope

The implemented-user-stories document defines 20 backend stories grouped by service. They cover registration, login, profile and progress; monster catalogue and collection; team creation and management; encounter discovery, calculation, outcomes, rewards and history; and service health.

Area	Stories	Main capabilities
User Service	US-01–US-04	Registration, login, JWT-based profile access and player progress.
Monster Service	US-05–US-08	Browse monsters, view collections, assign starter monsters and acquire monsters.
Team Service	US-09–US-12	Create teams, list teams, replace members and remove members.
Encounter Service	US-13–US-19	Browse encounters, resolve encounters, apply matchups/abilities, return results, grant rewards and show history.
Reliability	US-20	Health endpoints, Docker Compose orchestration and database health.

3.2 Key acceptance rules

Registration rejects duplicate usernames or email addresses and invalid account details. Successful login returns a JWT. Protected profile and progress operations require valid authentication. Teams contain one to three monsters, cannot contain duplicates, and may only use monsters owned by the player. Encounter outcomes are calculated by the backend rather than trusted from client input.

3.3 Non-functional concerns

The architecture guide identifies security, reliability and testability as important concerns: password hashing, authentication and ownership checks, backend input validation, parameterised database access, server-side calculation of team power and outcomes, consistent persistence of results and rewards, and keeping secrets out of source control.

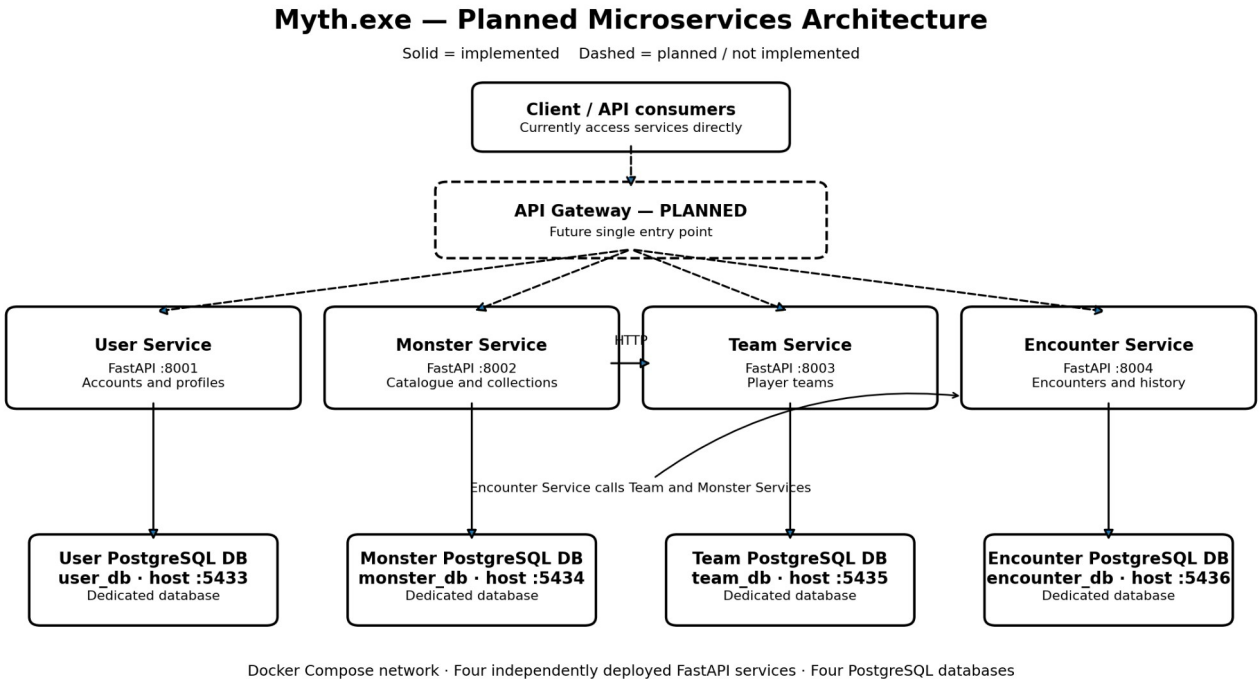
3.4 Scope boundaries

The current prototype does not include a graphical frontend, a running API Gateway, HTTPS deployment or a production security configuration. The Gateway OpenAPI file is a proposed contract only. The game-rules document is marked as proposed design; its contents should not all be assumed to be implemented.

4. SYSTEM ARCHITECTURE

4.1 High-level architecture

The system contains a client/API-consumer layer, four FastAPI services and four dedicated PostgreSQL databases. Clients currently access each service directly. The API Gateway is shown as a future single entry point and is represented with a dashed outline in the supplied diagram.



4.2 Deployment model

Docker Compose starts the four application services and their four PostgreSQL database containers. The services are published on host ports 8001–8004. The database ports published to the host are 5433–5436. Within the Docker network, services communicate through their Compose service names and container ports.

4.3 Service communication

The Team Service obtains monster information from the Monster Service to validate team members and calculate team power using current monster statistics. The Encounter Service calls the Team and Monster services to retrieve the selected team and monster data needed to resolve an encounter. The User Service interacts with the Monster Service during starter-collection assignment, as described in the user-story acceptance criteria.

4.4 Database isolation

Each service owns a separate PostgreSQL database. A developer's local Docker volumes remain local to that machine. Cloning the repository transfers source code and configuration templates, not another developer's live database records.

5. MICROSERVICES

Service	Host port	Database / host port	Responsibilities
User	8001	user_db / 5433	Registration, login, JWT authentication, profile and progress.
Monster	8002	monster_db / 5434	Catalogue, monster details, collections, starter assignment and acquisition.
Team	8003	team_db / 5435	Team creation/retrieval, membership validation, replacement/removal and power calculation.
Encounter	8004	encounter_db / 5436	Available encounters, battle calculation, outcome persistence, rewards and history.

5.1 User Service

The User Service provides player account operations. Its documented API includes registration, login, health checking and retrieval of the authenticated player's profile. The user-story document describes JWT access tokens and rejection of missing or invalid tokens for protected profile and progress requests.

5.2 Monster Service

The Monster Service exposes the monster catalogue and individual monster details and manages player collections. It supports starter-monster assignment and adding a monster to a player's collection. Duplicate ownership entries should be prevented, and invalid player or monster identifiers rejected.

5.3 Team Service

The Team Service creates and retrieves teams and supports changing team membership. A team must contain at least one and at most three monsters. A player can only select monsters in their collection, and the same monster cannot appear more than once. Team power is based on the sum of Attack and Defense values for the team's monsters. The current implementation has been described as recalculating team power from current monster catalogue values when teams are retrieved.

5.4 Encounter Service

The Encounter Service lists available wild-monster encounters, resolves an encounter using a selected player team, stores the result and exposes player encounter history. It retrieves authoritative team and monster data from the other services.
The backend calculates the values used to determine the outcome.

6. API DESIGN AND SERVICE COMMUNICATION

6.1 API style

The services expose HTTP APIs and exchange JSON data. FastAPI provides interactive Swagger UI and ReDoc documentation for each running service. OpenAPI JSON specifications are stored in the repository's docs directory.

Specification	Status
docs/user-service-openapi.json	Implemented service specification
docs/monster-service-openapi.json	Implemented service specification
docs/team-service-openapi.json	Implemented service specification
docs/encounter-service-openapi.json	Implemented service specification
docs/api-gateway-openapi.json	Proposed Gateway contract; Gateway not implemented

6.2 Interactive API documentation

When the application is running, Swagger UI is available at http://localhost:8001/docs, :8002/docs, :8003/docs and :8004/docs. ReDoc is available at the corresponding /redoc paths.

6.3 Planned API Gateway

The Gateway is intended to provide a central entry point that forwards requests to the appropriate backend service. The repository contains a proposed OpenAPI description for this future interface. It is not a running service and does not currently replace direct access to the four service ports.

7. GAME DOMAIN AND RULES

7.1 Monster roster

The Game Rules Specification for ten monsters. Each has a base stat budget of 150 points split between Attack and Defense.

Monster	Element	Attack	Defense	Role
Flameling	Fire	95	55	Attacker
Aquaris	Water	65	85	Defender
Stonehorn	Earth	55	95	Tank
Shadowling	Shadow	100	50	Special attacker
Leafiris	Nature	75	75	Balanced
Voltstrike	Electric	105	45	Fast attacker
Frostcore	Ice	70	80	Defensive control
Skyflare	Wind	60	90	Defensive support
Magmalius	Lava	90	60	Heavy attacker
Lumira	Light	80	70	Balanced support

7.2 Elemental matchups

Element	Strong against	Weak against
Fire	Earth	Water
Water	Fire	Earth
Earth	Water	Fire
Shadow	Light	Nature
Nature	Shadow	Lava
Electric	Water	Wind
Ice	Wind	Light
Wind	Electric	Ice
Lava	Nature	Electric
Light	Shadow	Ice

The proposed matchup modifier is +20% Attack for an advantage, −20% Attack for a disadvantage, and no change for a neutral matchup.

7.3 Automatic special abilities

Monster	Ability	Proposed effect
Flameling	Blaze Surge	+15% Attack against Earth
Aquaris	Tidal Shield	+15% Defense
Stonehorn	Granite Guard	+15% Defense

Shadowling	Umbral Strike	Ignores elemental disadvantages
Leafiris	Healing Bloom	+10% total team power
Voltstrike	Thunder Burst	+15% Attack against Water
Frostcore	Frost Bind	+15% Defense against Fire
Skyflare	Gale Rush	+10% Attack against Lava
Magmalius	Molten Impact	+15% Attack against Nature
Lumira	Radiant Ward	+10% total team power

The game-rules specification describes these abilities as automatic deterministic modifiers rather than turn-based effects such as stun, healing or extra attacks. It says percentage-modified individual statistics are rounded to the nearest integer before calculating the team total.

7.4 Encounter calculation and specification difference

The implemented-user-stories document states that prototype difficulty is (wild monster Attack + Defense) multiplied by the number of monsters in the player's team. The separate Game Rules Specification instead defines difficulty as the wild monster's Attack + Defense without multiplying by team size

Both documents define success as Team Power greater than or equal to difficulty; otherwise the result is FAILURE. The user-story document describes every resolved encounter being saved and successful encounters triggering acquisition of the wild monster.

7.5 Team-wide bonus stacking

The game-rules specification says Leafiris and Lumira each provide a +10% total team-power bonus, but explicitly records stacking of these bonuses as an open decision. The project should document and test the selected behaviour before finalising the mechanic.

8. IMPLEMENTATION STATUS

The following status reflects the supplied implemented-user-stories document, architecture material and reported prototype state. Planned items are not claimed as completed.

Capability	Status	Notes
Registration and login	Implemented	Account validation and JWT-based login are documented.
Profile and progress	Backend prototype	Protected profile/progress operations are described.
Monster catalogue and collection	Implemented	Catalogue, collection retrieval, starter assignment and acquisition.
Team management	Implemented	Create/list/update membership, remove members and calculate power.
Encounter discovery and resolution	Implemented	Backend calculates battle values and stores outcomes.
Elemental and special abilities	Confirm exact deployed rules	Rules PDF is marked proposed; verify exact code version.
Rewards and history	Implemented	Successful encounters request acquisition; resolved encounters are stored.
Health endpoints and Compose	Implemented	Four services and four PostgreSQL containers.
API Gateway	Planned	OpenAPI contract exists; Gateway service is not implemented.
Frontend	Not implemented	Clients access service APIs directly.
HTTPS / production hardening	Not implemented	Future development task.

9. TESTING AND VERIFICATION

9.1 Automated tests

The latest supplied test evidence reports 51 tests passed and zero failures. The test suite is organised around the four services and covers registration/login, monster catalogue and collection behaviour, team management and power calculation, encounter calculations, outcomes, rewards and history.

9.2 API and Docker checks

The four service health endpoints are /health on ports 8001 through 8004. Docker-based checks were used to verify service startup and API behaviour. The project history also records a successful encounter response containing calculated team power, difficulty and outcome.

9.3 Test commands

```
python -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
python -m pytest -v
```

10. SECURITY, RELIABILITY AND DATA MANAGEMENT

The architecture guide identifies these security controls: hash passwords using a modern password-hashing algorithm; require authentication for player-specific endpoints and check ownership; validate all backend input; use ORM features or parameterised queries; calculate team power, outcomes and rewards on the server; persist outcomes and rewards consistently, using database transactions where needed; and keep secrets out of source control.

Docker Compose health checks and service health endpoints provide basic operational visibility. Separate databases support service-level data ownership. Production readiness would additionally require security review, TLS/HTTPS, secret management, access-control verification, backup/restore procedures and monitoring.

11. MAIN API ENDPOINTS

Service	Method	Endpoint	Purpose
User	GET	/health	Service health
User	POST	/register	Register player
User	POST	/login	Authenticate player
User	GET	/me	Authenticated profile
Monster	GET	/health	Service health
Monster	GET	/monsters	List monster catalogue
Monster	GET	/monsters/{monster_id}	Get monster details
Monster	GET	/players/{player_id}/collection	Get player collection
Monster	POST	/players/{player_id}/collection	Add monster to collection
Team	GET	/health	Service health
Team	GET	/players/{player_id}/teams	List player teams

Team	POST	/players/{player_id}/teams	Create team
Encounter	GET	/health	Service health
Encounter	GET	/encounters/available	List available encounters
Encounter	POST	/encounters	Resolve encounter
Encounter	GET	/players/{player_id}/encounters	Get encounter history

12. FULL USER STORIES AND ACCEPTANCE CRITERIA

C.1 User Service

US-01: Player Registration

As a new player, I want to create an account using my username, email and password, so that I can start playing Myth.exe.

Acceptance criteria

- The system allows registration with valid account information.
- An already registered username or email address is rejected.
- The email format is validated.
- Passwords shorter than the configured minimum length are rejected.
- After successful registration, the player receives the starter monster collection.
- The system handles Monster Service unavailability during registration.

US-02: Player Login

As a registered player, I want to log in using my credentials, so that I can authenticate myself and access my account.

Acceptance criteria

- Valid credentials allow login.
- An incorrect password is rejected.
- Login attempts for unknown usernames are rejected.
- Successful authentication returns a JWT access token.

US-03: View Player Profile

As a registered player, I want to view my profile, so that I can access my account information.

Acceptance criteria

- A valid JWT allows the player to retrieve their profile.
- Requests without a token are rejected.
- Requests with an invalid token are rejected.
- Authenticated requests return profile information.

US-04: View Player Progress

As a registered player, I want to view my game progress, so that I can monitor my activity in Myth.exe.

Acceptance criteria

- The player can request progress using a valid JWT.
- Requests without a valid token are rejected.
- The system retrieves relevant player progress information.
- The system handles Monster Service unavailability appropriately.

C.2 Monster Service

US-05: Browse Available Monsters

As a player, I want to browse the available monsters, so that I can discover the monsters in Myth.exe.

Acceptance criteria

- The system returns the available monster catalogue.
- Each monster includes its ID, name, element/type, Attack and Defense statistics.
- The system supports retrieving an individual monster by ID.
- Requests for unknown monster IDs return an appropriate error.

US-06: View My Monster Collection

As a player, I want to view my monster collection, so that I can see which monsters I own.

Acceptance criteria

- The system returns the monsters belonging to the requested player.
- Each collection entry includes the monster's details.
- A player can own multiple monsters.

US-07: Receive Starter Monsters

As a newly registered player, I want to receive an initial monster collection, so that I can begin playing without an empty collection.

Acceptance criteria

- The system assigns the initial monsters to the player.
- Starter monsters are assigned during registration.
- Repeating the starter assignment does not create duplicate collection entries.
- Invalid player IDs are rejected.

US-08: Acquire a Monster

As a player, I want to acquire a new monster after a successful encounter, so that I can expand my collection.

Acceptance criteria

- The system allows an existing monster to be added to a player's collection.
- Unknown monster IDs and invalid player IDs are rejected.
- Duplicate ownership entries are prevented.
- The Encounter Service requests acquisition after a successful battle.

C.3 Team Service

US-09: Create a Monster Team

As a player, I want to create a team using monsters from my collection, so that I can prepare a team for encounters.

Acceptance criteria

- A player can create a team using owned monsters.
- A team must contain at least one monster and at most three monsters.
- Duplicate monsters are not allowed in the same team.
- Players cannot add monsters they do not own.
- Base team power is the sum of the Attack and Defense statistics of its monsters.

US-10: View My Teams

As a player, I want to view my existing teams, so that I can check their members and power.

Acceptance criteria

- The system returns all teams belonging to the requested player.
- Each team includes its ID, player ID, monster IDs and team power.

- Team power is calculated using current monster statistics.
- The system handles Monster Service unavailability appropriately.

US-11: Replace a Monster in a Team

As a player, I want to replace a monster in my team with another monster I own, so that I can change my team composition.

Acceptance criteria

- The player can replace an existing team member.
- The replacement monster must belong to the player's collection.
- A monster already present in the team cannot be added again.
- The system recalculates base team power after replacement.
- Invalid replacement requests return an appropriate error.

US-12: Remove a Monster from a Team

As a player, I want to remove a monster from my team, so that I can modify my team composition.

Acceptance criteria

- The player can remove a monster from an existing team.
- The system updates team membership and recalculates base power.
- A player cannot remove the last remaining monster from a team.

C.4 Encounter Service

US-13: Browse Available Encounters

As a player, I want to view available wild monsters for encounters, so that I can choose an opponent to battle.

Acceptance criteria

- The system retrieves available monsters from the Monster Service.
- Each available encounter includes wild monster ID, name, element/type and difficulty.
- Encounter difficulty is calculated by the backend.
- The current prototype calculates difficulty as (wild monster Attack + Defense) multiplied by the number of monsters in the player's team.

US-14: Start a Monster Encounter

As a player, I want to start an encounter using one of my teams, so that I can battle a wild monster.

Acceptance criteria

- The player provides a player ID, team ID and wild monster ID.
- The system validates the requested player and team.
- The system retrieves team members and their monster statistics.
- The system calculates battle power on the backend.
- The system does not rely on a player-supplied difficulty value.
- The outcome is determined by comparing team power with encounter difficulty.

US-15: Apply Elemental Matchups

As a player, I want to benefit from elemental advantages and disadvantages during battles, so that monster selection becomes strategically meaningful.

Acceptance criteria

- The battle calculation considers each team monster's elemental relationship with the wild monster.

- An elemental advantage increases Attack by 20%.
- An elemental disadvantage decreases Attack by 20%.
- Neutral matchups do not modify Attack.
- The configured elemental matchup rules are applied.

US-16: Activate Monster Special Abilities

As a player, I want to benefit from my monsters' special abilities during encounters, so that each monster can contribute a unique advantage.

Acceptance criteria

- Special abilities activate automatically when their configured conditions are met.
- Abilities modify Attack, Defense or total team power according to their definitions.
- Shadowling ignores elemental disadvantages.
- Leafiris and Lumira provide team-wide power bonuses.
- Multiple team-wide bonuses are added together.
- Percentage modifiers are combined before resulting statistics are rounded to the nearest integer.

US-17: Receive an Encounter Result

As a player, I want to receive the result of my encounter, so that I know whether my team defeated the wild monster.

Acceptance criteria

- The system compares calculated team power against encounter difficulty.
- An encounter succeeds when team power is greater than or equal to difficulty.
- An encounter fails when team power is lower than difficulty.
- The response includes team power, difficulty, wild monster ID and outcome.
- The encounter result is saved in the database.

US-18: Receive a Monster Reward

As a player, I want to receive the wild monster after a successful encounter, so that I can expand my collection through battles.

Acceptance criteria

- A successful encounter triggers monster acquisition.
- The acquired monster is added to the player's collection.
- A failed encounter does not trigger monster acquisition.
- Existing ownership is handled without duplicate collection entries.

US-19: View Encounter History

As a player, I want to view my previous encounters, so that I can review my battle results.

Acceptance criteria

- The system retrieves encounter records for the requested player.
- Each record includes encounter ID, player ID, team power, difficulty, wild monster ID, outcome and creation timestamp.
- Both successful and failed encounters are stored.
- Previous records remain available after new encounters are completed.

C.5 Service Health and Reliability

US-20: Check Microservice Health

As a system operator, I want to check the health of each microservice, so that I can verify that backend services are running.

Acceptance criteria

- Each microservice exposes a health endpoint.
- The health endpoint returns a healthy status when the service is operational.
- The four services can be started and managed using Docker Compose.
- PostgreSQL database containers report healthy status when operational.